

Sistemas Distribuídos

Licenciatura em Engenharia Informática

Trabalho Prático 1

Sistema Distribuído de Gestão



UNIVERSIDADE DE ÉVORA

Miguel Pombeiro, 57829 | Miguel Rocha, 58501

Índice

1	Introdução	2
2	Escolhas realizadas ao longo do trabalho	2
2.1	Base de Dados	3
3	Instruções	4
4	Desafios encontrados	5

1 Introdução

Para este trabalho, foi proposto o desenvolvimento de um Sistema Distribuído de Gestão. Para tal, foi escolhido como domínio um Sistema de Gestão de Reservas de Apartamentos, inspirado no modelo do Airbnb. Assim sendo, foi construída uma base de dados relacional em PostgreSQL, um servidor em Java, que tem como funcionalidades a consulta e alteração de entidades e dois clientes que foram implementados com recurso a tecnologias de comunicação diferentes. O cliente geral, que faz as operações típicas de utilizador foi implementado com recurso a *sockets* e classes `Java.io.Serializable` e o cliente administrador foi implementado com recurso a Java RMI.

2 Escolhas realizadas ao longo do trabalho

Ao longo do desenvolvimento do sistema, foram tomadas várias decisões. Estas decisões foram importantes para o rumo do projeto, destacando-se as seguintes:

- A escolha de *sockets* com `Java.io.Serializable` para o cliente geral prendeu-se com a nossa familiaridade com esta tecnologia, sendo necessário implementar muitas mais funcionalidades neste cliente, esta pareceu a opção mais indicada. A escolha de RMI para as funcionalidades administrativas permite utilizar uma tecnologia mais tolerante a falhas e com uma maior segurança para operações mais sensíveis.
- Realização de uma autenticação simples no cliente geral para limitar o acesso do utilizador a operações sensíveis (e.g. Aceder a apartamentos ou reservas de outros utilizadores). Este processo, permite também à *thread* responsável por cada cliente poder identificá-lo para a realização de operações na base de dados.
- Utilização de um ficheiro de configuração (`.properties`) para parametrizar ligação à base de dados bem como do servidor. Desta forma, é possível deixar o código mais modular, sendo apenas necessário alterar parâmetros essenciais ao bom funcionamento em apenas um local.
- Representação de cada entidade por classes DTO (Data Transfer Objects) que implementam `java.io.Serializable` para permitir a serialização na troca de pedidos/resposta e facilitar a comunicação cliente-servidor.
- Iniciar e encerrar da ligação à base de dados nas operações remotas para reduzir o risco de conexões persistentes.
- Estruturação do código em várias classes para melhorar a legibilidade e modularidade do mesmo, facilitando assim a adição de futuras novas funcionalidades.
- Validação ao inserir/alterar dados na base de dados, apenas no lado do servidor e do cliente, em vez de confiar apenas em verificações na base de dados, tendo assim mais controlo sobre os dados a inserir.

- Limitação das alterações que o administrador pode realizar em certos atributos, para diminuir a probabilidade de introduzir inconsistências acidentais na base de dados.

2.1 Base de Dados

A nossa base de dados relacional em PostgreSQL foi uma das escolhas mais importantes para este projeto, que foi cuidadosamente planeada de modo a ter todos os dados relevantes para a representação das entidades principais do sistema. Após uma cuidada análise ao enunciado e do nosso domínio, decidimos criar três entidades, sendo estas utilizadores (users), apartamentos (apartments) e reservas (reservations), representadas da seguinte maneira:

```
1 CREATE TABLE users (  
2     user_id SERIAL NOT NULL,  
3     name VARCHAR(100) NOT NULL,  
4     email VARCHAR(100) UNIQUE NOT NULL,  
5     phone VARCHAR(20) NOT NULL,  
6     operational_state VARCHAR(30) NOT NULL,  
7     admin_state VARCHAR(30) NOT NULL,  
8     created_at TIMESTAMP NOT NULL,  
9     PRIMARY KEY (user_id)  
10 );
```

```
1 CREATE TABLE apartments (  
2     apartment_id SERIAL NOT NULL,  
3     name VARCHAR(100) NOT NULL,  
4     location VARCHAR(100) NOT NULL,  
5     price_per_night DECIMAL(10,2) NOT NULL,  
6     type INT NOT NULL,  
7     operational_state VARCHAR(30) NOT NULL,  
8     admin_state VARCHAR(30) NOT NULL,  
9     owner_id INT NOT NULL,  
10    PRIMARY KEY (apartment_id)  
11 );
```

```
1 CREATE TABLE reservations (  
2     reservation_id SERIAL NOT NULL,  
3     apartment_id INT NOT NULL,  
4     renter_id INT NOT NULL,  
5     start_date DATE NOT NULL,  
6     end_date DATE NOT NULL,  
7     total_price DECIMAL(10,2) NOT NULL,  
8     operational_state VARCHAR(30) NOT NULL,  
9     admin_state VARCHAR(30) NOT NULL,
```

```
10 |     created_at TIMESTAMP NOT NULL,  
11 |     PRIMARY KEY (reservation_id)  
12 | );
```

Cujas chaves estrangeiras estão refletidas nas seguintes linhas de código:

```
1 | ALTER TABLE reservations ADD FOREIGN KEY (apartment_id) REFERENCES  
  | apartments(apartment_id) ON DELETE RESTRICT;  
2 | ALTER TABLE reservations ADD FOREIGN KEY (renter_id) REFERENCES  
  | users(user_id) ON DELETE RESTRICT;  
3 | ALTER TABLE apartments ADD FOREIGN KEY (owner_id) REFERENCES users(user_id)  
  | ON DELETE RESTRICT;
```

3 Instruções

De modo a utilizar este serviço é necessário fazer os passos descritos adiante, tendo também em conta o conteúdo do ficheiro `config.properties` e os respetivos ficheiros para o docker, `.env` e `docker-compose.yml` :

1. Inicializar o Docker, usando o comando na diretoria base do projeto

```
1 | docker compose up
```

2. Criar uma Base de Dados e conectar para o PostgreSQL.

2.1. Abrir `http://localhost:16543`

2.2. *Login* com as credenciais do ficheiro `.env`

- **Email:** admin@admin.com
- **Password:** admin123

2.3. Registrar um novo servidor no pgAdmin:

- **Host name/address:** postgres
- **Port:** 5432
- **Database:** gestao_apartamentos
- **Username:** trabalho1
- **Password:** sistemasDistribuidos

3. Na base de dados criada, inserir o código presente no ficheiro `databases/table_creation.sql`. Na mesma diretória, acompanha um ficheiro `testing.sql` que pode ser útil para popular as tabelas;

4. Compilar o código com a ajuda da Makefile, com

```
1 | make compile
```

5. Executar o servidor num terminal da pasta do projeto

```
1 | make run-server
```

6. Executar o cliente administrador e o cliente geral, em diferentes terminais da pasta do projeto

```
1 | make run-admin-client  
2 | make run-general-client
```

4 Desafios encontrados

Ao longo do desenvolvimento do sistema, o grupo foi encontrando vários desafios que permitiram aprimorar as nossas competências.

A primeira dificuldade encontrada e, talvez a maior, foi a compreensão do enunciado e daquilo que era necessário implementar. O enunciado do trabalho era de alto nível e deixava muita margem para interpretação, pelo que foi necessário delinear e clarificar o seu domínio: que entidades deveriam existir, quais as operações que ambos os clientes deveriam suportar, de modo a manter o sistema consistente, e como seria o fluxo completo de interação (desde a interface de acesso no terminal à base de dados). Desta forma, foi necessário transformar as ideias gerais transmitidas no enunciado em requisitos mais concretos e definir prioridades sobre o que desenvolver.

A nível de outros sistemas utilizados, o Docker levantou algumas dúvidas, nomeadamente ao nível da integração com a aplicação Java que estávamos a desenvolver. Assim sendo, optamos por utilizar apenas a configuração que exploramos nas aulas práticas com os serviços PostgreSQL e pgAdmin para aceder e manipular diretamente a base de dados, deixando o ambiente Java da aplicação de fora.

A nível do cliente geral, implementado com *sockets*, não tivemos dificuldades de maior, uma vez que é uma tecnologia que já tinha sido mais explorada em Redes de Computadores. A maior novidade introduzida, neste campo, foi a utilização de objetos serializados para a troca de informação entre o servidor e os clientes, contudo, a sua utilização acabou por facilitar a nossa implementação.

O RMI, utilizado no cliente administrativo, acabou por representar o maior desafio do ponto de vista de implementação do serviço. Compreender o registo no *RMI Registry*, o *binding* de objetos remotos e a necessidade de definir o `java.rmi.server.hostname` em alguns ambientes levou algum tempo. Ocorreram algumas situações em que o cliente não conseguia localizar o *stub* devido a questões de *hostname* e *firewall*.

Depois de superadas as maiores dificuldades nomeadamente um traduzir o enunciado para requisitos e a curva de aprendizagem para a utilização de RMI, achamos ter conseguido desenvolver um sistema que implementa as funcionalidades pretendidas.